VECTOR

# vHsmUpd

## Technical Reference

### HSM Update Process

Version 3.0.3


| | |
|---|---|
| Authors | Tobias Schwindl, Bethina Mausz, Susanne Wellnhofer |
| Status | Released |

# Document Information

## History

| Author | Date | Version | Remarks |
|---|---|---|---|
| Tobias Schwindl | 2019-02-18 | 1.00.00 | Initial creation of the Technical Reference for the vHsmUpd. |
| Tobias Schwindl, Bethina Mausz | 2019-03-22 | 1.01.00 | STORY-10323: vHsmUpd shall search at multiple addresses for an update header.<br>STORY-8706: vHsmUpd shall inform Host-Core about pending update |
| Tobias Schwindl | 2019-04-25 | 2.00.00 | STORY-10995: vHsmUpd shall use Core/LL concept to minimize development effort for new platforms |
| Tobias Schwindl | 2019-05-14 | 2.01.00 | STORY-11970: vHsmUpd shall support startup of Host-Cores |
| Tobias Schwindl | 2019-06-04 | 2.01.00 | STORY-9512: vHsmUpd shall use new memory interface (vMemAccM and vMem) |
| Tobias Schwindl | 2019-07-19 | 2.02.00 | MWDG-40: vHsmUpd shall use write/erase retry strategy of vMemAccM |
| Tobias Schwindl | 2019-08-05 | 2.02.01 | MWDG-1569: HSM Updater solution shall take care of RAM alignment of data buffers |
| Susanne Wellnhofer | 2019-08-28 | 3.00.00 | MWDG-94: HSM Updater shall evaluate execution reason to decide for update request |
| Tobias Schwindl | 2019-09-26<br>2019-11-06 | 3.00.01 | MWDG-1755: Support RH850 P1x-C devices<br>ESCAN00104801: HSM Updater can give unaligned write buffers to underlying Memory Abstraction module which leads to exception |
| Tobias Schwindl | 2019-11-13 | 3.00.02 | MWDG-1545: HSM Update shall support Traveo II devices |
| Tobias Schwindl | 2020-03-02 | 3.00.03 | ESCAN00105693: HSM Updater documentation does not describe which part of solution shall be mapped to RAM to function correctly |

## Reference Documents

| No. | Source | Title | Version |
|---|---|---|---|
| [1] | Vector | TechnicalReference_Crypto_30_vHsm.pdf | See delivery |
| [2] | Vector | TechnicalReference_Crc.pdf | See delivery |
| [3] | Vector | TechnicalReference_vMemAccM.pdf | See delivery |
| [4] | Vector | TechnicalReference_vMem.pdf | See delivery |
| [5] | Vector | TechnicalReference_vMem_xxx.pdf | See delivery |



**Caution**
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2020 Vector Informatik GmbH

## Contents

**1 Component History ..... 7**

**2 Introduction ..... 8**
2.1 Architecture Overview ..... 8

**3 Functional Description ..... 9**
3.1 Features ..... 9
3.2 Functional Overview ..... 9
3.2.1 Memory Layout ..... 9
3.2.2 Workflow ..... 10
3.3 Initialization ..... 11
3.4 Communication with host core ..... 12
3.5 Communication and interaction with Host system ..... 13
3.6 Main Functions ..... 14
3.7 Structure of Update Header ..... 14
3.8 Structure of Update Image ..... 15

**4 Integration ..... 17**
4.1 Scope of Delivery ..... 17
4.1.1 Static Files ..... 17
4.1.2 Dynamic Files ..... 17
4.2 MemMap Sections used by vHsmUpd ..... 18
4.3 Memory Layout of Secure Code Flash for HSM Firmware ..... 18
4.4 Device Startup ..... 19
4.5 Interrupt Vector Table ..... 20
4.6 Integration of Memory Stack for Secure Code Flash Access ..... 20

**5 API Description ..... 23**
5.1 Type Definitions ..... 23
5.2 Services used by vHsmUpd ..... 23
5.3 Callout Functions ..... 24
5.3.1 vHsm_FirmwareUpdate_GetMacKey_Callout ..... 24
5.3.2 vHsm_FirmwareUpdate_GetCipherKey_Callout ..... 24
5.3.3 vHsm_FirmwareUpdate_Error_Callout ..... 25
5.3.4 vHsm_FirmwareUpdate_VersionCompare_Callout ..... 26
5.3.5 vHsm_FirmwareUpdate_Update_Callout ..... 26
5.3.6 vHsmUpd_FirmwareUpdate_TriggerSoftwareReset_Callout ..... 27

**6 Configuration ..... 29**

6.1 Configuration Variants ..... 29
6.2 Configuration with DaVinci Configurator Pro ..... 29
6.2.1 vHsmUpd Header ..... 29
6.2.2 vHsmUpd Retry Strategy using vHsmUpdRetryCounter ..... 29
6.2.3 vMemAccM and vMem Configuration Restrictions ..... 30
6.3 Configuration of Hardware Abstraction Layer (HAL) ..... 31

**7 Additional Hints and Recommendations Testing the vHsmUpd ..... 32**

**8 Glossary and Abbreviations ..... 33**
8.1 Glossary ..... 33
8.2 Abbreviations ..... 33

**9 Contact ..... 34**

## Illustrations

Figure 2-1 AUTOSAR 4.3 Architecture Overview ..... 8
Figure 3-1 Abstract memory layout ..... 9
Figure 3-2 Update header format ..... 15
Figure 3-3 Update container with embedded update image (encrypted parts) ..... 16
Figure 4-1 Memory Layout for Hsm Updater manage secure code area ..... 18

## Tables

Table 1-1 Component history ..... 7
Table 3-1 Provided Features ..... 9
Table 3-2 Update Header ..... 14
Table 3-3 Presence Pattern ..... 15
Table 3-4 Update Image ..... 16
Table 4-1 Static files ..... 17
Table 4-2 Generated files ..... 17
Table 4-3 Used non-standard MemMap sections ..... 18
Table 5-1 Type definitions ..... 23
Table 5-2 Services used by the vHsmUpd ..... 23
Table 5-3 vHsm_FirmwareUpdate_GetMacKey_Callout ..... 24
Table 5-4 vHsm_FirmwareUpdate_GetCipherKey_Callout ..... 25
Table 5-5 vHsm_FirmwareUpdate_Error_Callout ..... 25
Table 5-6 vHsm_FirmwareUpdate_VersionCompare_Callout ..... 26
Table 5-7 vHsm_FirmwareUpdate_Update_Callout ..... 27
Table 5-8 vHsmUpd_FirmwareUpdate_TriggerSoftwareReset_Callout ..... 27
Table 6-1 vMemAccM configuration ..... 30
Table 8-1 Glossary ..... 33
Table 8-2 Abbreviations ..... 33

# 1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

| Component Version | New Features |
|---|---|
| 1.00.00 | Initial creation of vHsmUpd. |
| 1.01.00 | Adds additional header search mechanism.<br>Adds information about communication with Host-Core. |
| 2.00.00 | Introduce of HighLevel/LowLevel concept into vHsmUpd. |
| 2.01.00 | Support to start Host before the update process is started.<br>Use of new internal memory architecture (vMem(AccM)). |
| 2.02.00 | Reworked update retry mechanism. |
| 3.00.00 | Evaluate execution reason to decide on update request. |

Table 1-1 Component history

# 2 Introduction

This document describes the functionality, API and configuration of the MICROSAR BSW module vHsmUpd.

| | | |
|---|---|---|
| **Supported AUTOSAR Release:** | 4.3.0 | |
| **Supported Configuration Variants:** | pre-compile | |
| **Vendor ID:** | VHSMUPD_VENDOR_ID | 30 decimal<br>(= Vector-Informatik, according to HIS) |
| **Module ID:** | VHSMUPD_MODULE_ID | 255 decimal |

The vHsmUpd manages the vHsm firmware image. This includes an update mechanism of the vHsm software. If no update is requested the vHsmUpd just starts the vHsm firmware.

## 2.1 Architecture Overview

The following figure shows where the vHsmUpd is located in the AUTOSAR architecture.

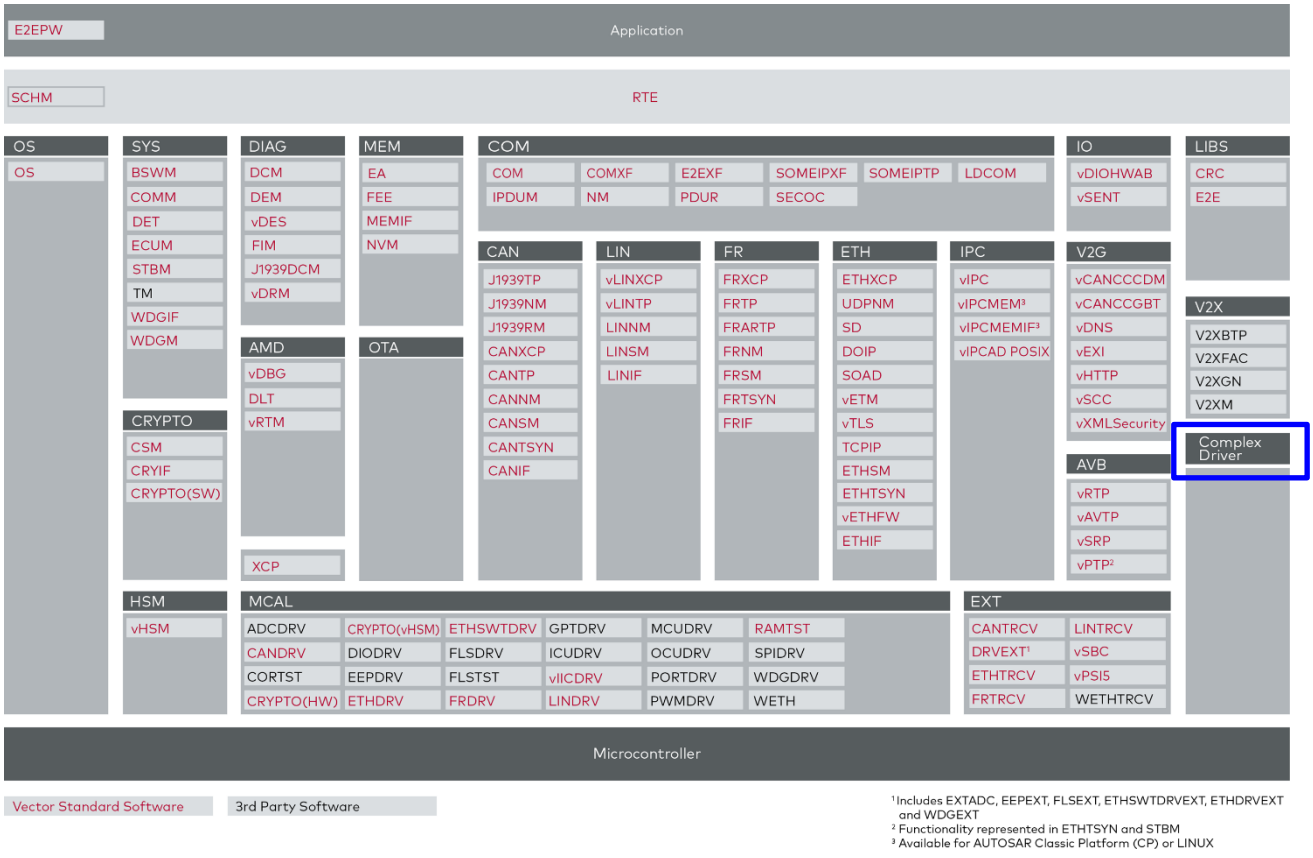



Figure 2-1 AUTOSAR 4.3 Architecture Overview

# 3 Functional Description

## 3.1 Features

The vHsmUpd provides the main features listed in the following table. Since the vHsmUpd is not an AUTOSAR component all features provided are beyond AUTOSAR standard:

| Provided Features |
|---|
| Starting the vHsm firmware if no update is requested |
| Execute an update request on the vHsm firmware |
| Provide information about the current vHsmUpd state to Host |
| Recovery mode, i.e. vHsm firmware is invalid before start of update |

Table 3-1 Provided Features

## 3.2 Functional Overview

### 3.2.1 Memory Layout

The following figure shows the abstract memory layout.

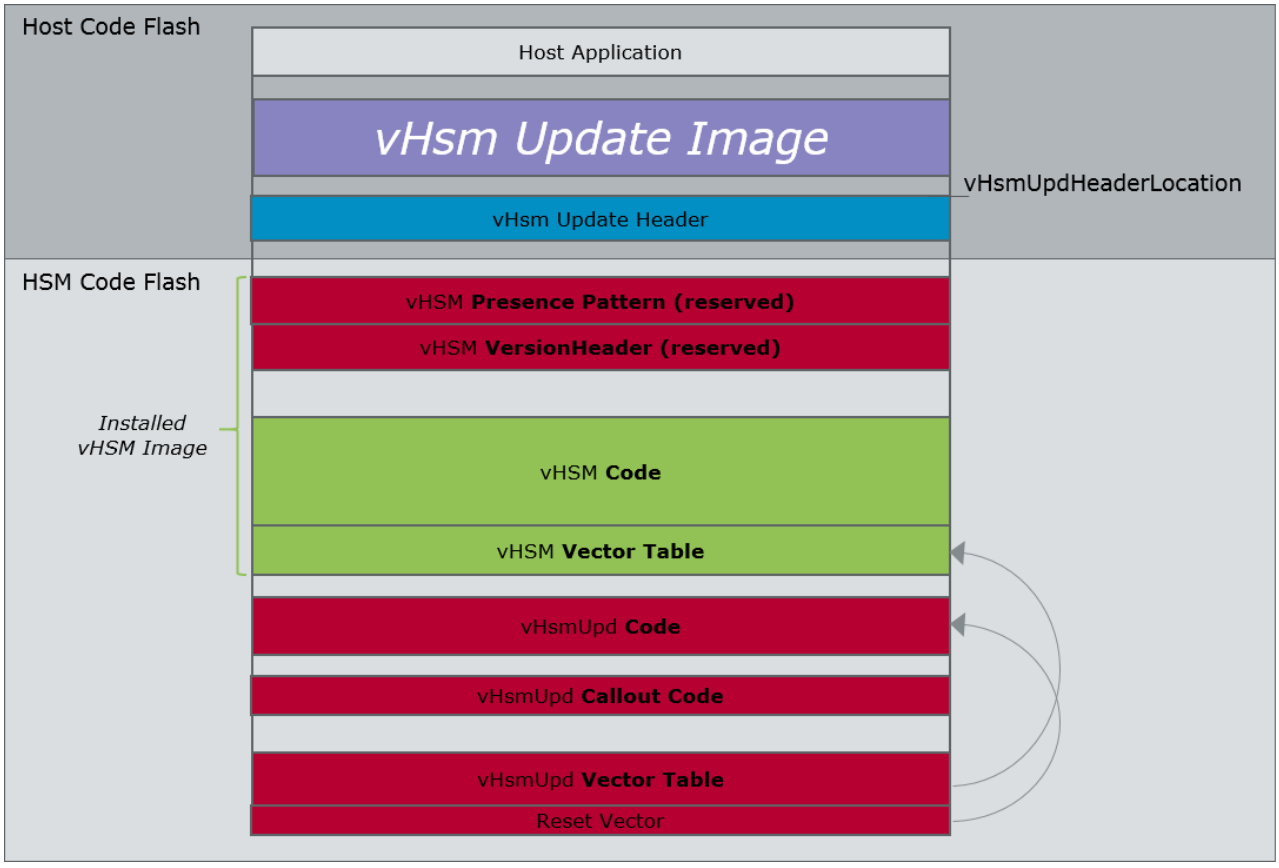



Figure 3-1 Abstract memory layout

- There is a Host Code Flash, from which the Host Application is executed, and a HSM Code Flash, from which the vHsm (aka Firmware) and the vHsmUpd are executed. HSM Code Flash is only accessible from CPU's HSM Core.
- In Host Code Flash there is a dedicated memory section, into which the vHsm Update Image can be written.
- In addition, there is a dedicated memory section, into which the vHsm Update Header can be written. A valid Update Header indicates that the currently installed firmware must be updated with the new Update Image.
- In HSM Code Flash there is a dedicated memory section for the vHsmUpd and the vHsm firmware. The vHsmUpd must be initially available and cannot be overwritten.
- The vHsmUpd shall be extended by the Startup code and by the Interrupt Vector Table. The Interrupt Vector Table forwards each interrupt to the firmware via the vHsm Vector Table. Only the Reset Vector will be forwarded to vHsmUpd.
- The vHsmUpd provides special Callout Code sections. Into these sections the callout functions may be mapped, which are called to retrieve keys for MAC generation and Image Encryption.
- The vHsm firmware reserves some special memory area for vHsm Presence Pattern as well as for vHsm Version Header. Both are written and checked by vHsmUpd and must be located directly before the vHsm Code area.

### 3.2.2 Workflow

From the vHsmUpd point of view the update process looks as described below. Depending on the status of the vHsm firmware upon start-up of the device, two scenarios are distinguished:

1. The current vHsm firmware is valid. In this scenario, the following steps are performed:

- If the device was recently reset, start the vHsm firmware directly.
  - Otherwise, a jump from the vHsm firmware to the vHsmUpd occurred. Then, if a valid update header is found, the vHsmUpd
    - processes the update header to determine if and where an image is available,
    - verifies the update image in Host memory,
    - performs an update if the new firmware is valid, and
    - starts the vHsm firmware.
  - If no valid update header is found, the vHsm firmware is started.

2. The current vHsm firmware is not valid, i.e. the device was started in recovery mode. In this scenario, the following steps are performed:

- If a valid update header is found, the vHsmUpd starts the Host system.

- The vHsmUpd waits until the Host application initialized the shared memory,
  - verifies the update image in Host memory,
  - performs an update if the new firmware is valid, and
  - triggers a reset of the device.
- If no valid update header was found, vHsmUpd triggers an error callout (see section 5.3.3).



**Note**
If the used hardware abstraction layer does not support the reset, vHsmUpd calls a corresponding reset callout as described in section 5.3.6.

## 3.3 Initialization

The vHsmUpd does not provide any initialization routine. In contrast to most other modules the vHsmUpd implements the `main()` function, in which all necessary software-initialization is executed.

Within the `main()` function vHsmUpd:

- looks for valid vHsm image within HSM code memory,
- checks for a possible update image within Host memory and finally
- decides either to start the installed firmware or to update the existing firmware.



**Note**
An update of the existing firmware is only performed if device was started in recovery mode or a direct jump from the vHsm firmware to the vHsmUpd occurred. The vHsm firmware is **not** updated directly after a reset of the device but started directly.



**Note**
The vHsmUpd initializes all used components itself, e.g. vMem and vMemAccM.

## 3.4 Communication with host core

For the communication with the host core a shared-memory based communication (Ipc-Communication) is used. The shared-memory contains a data structure for the Hsm-to-Host communication and, if configured, a data structure for vHsmUpd states.

1. Hsm-to-Host communication:

   The address of the shared memory for the Hsm-to-Host communication has to be writable by the Hsm and readable by the Host application.

   The vHsmUpd checks the bit 0 of this shared memory address to evaluate whether a jump from the Hsm firmware occurred (bit 0 is set) or the device was just reset (bit 0 is cleared). In case a jump occurred, the vHsmUpd clears both bit 0 and bit 1 before checking the update request.



**Note**
Bit0 represents the “Hsm started” information while Bit1 represents the “Hsm ready for operation” information. Both are only set by the Hsm firmware.

   Additionally, at startup, the vHsmUpd sets bit 15 to indicate that it is running and using this shared memory address. The vHsmUpd clears the bit 15 right before starting the Hsm firmware.

2. vHsmUpd states (if configured):

   The address of the shared memory for the vHsmUpd states has to be writable by the vHsmUpd and should be only readable by the host, as there is no synchronization mechanism.

   For example, while the vHsmUpd is performing an update, the host core may want to check the state of vHsm, therefore vHsmUpd sets a corresponding state in shared memory. To indicate a pending update `VHSMUPD_UPDATE_IN_PROGRESS` is set. Also, the states `VHSMUPD_UPDATE_NOT_STARTED` and `VHSMUPD_UPDATE_COMPLETED` are set by vHsmUpd. The `VHSMUPD_UPDATE_NOT_STARTED` state is used for all errors that occur before the flash content is changed. Otherwise, the state `VHSMUPD_UPDATE_IN_PROGRESS` is used.



**Caution**
In the case of communication of the vHsmUpd states, vHsmUpd uses just one byte.

The addresses of shared memory can be configured via the corresponding `vHsmUpdIpc` configuration parameters.



**Caution**
Hardware restrictions regarding the addresses for communication with the host core have to be respected by the user, i.e. alignment and/or locations have to be chosen correctly.

## 3.5 Communication and interaction with Host system

If the vHsmUpd was started in recovery mode, see 3.2.2, the vHsmUpd has to wait until the Host application initialized the shared memory.

To achieve this, the vHsmUpd relies on the data structure for the Host-to-Hsm communication which is located in the shared memory. Accordingly, this address of the shared memory for the Host-to-Hsm communication has to be writable by the Host and readable by the Hsm application. The vHsmUpd has to be able to evaluate this address in order to proceed according to 3.2.2.

In detail, the vHsmUpd checks whether the mask parameter `WaitMask`, a `vHsmUpdIpc` configuration parameter, is set (host already initialized the shared memory) or not.



**Caution**
If the `WaitMask` is never set, the device is bricked! This happens since the vHsmUpd waits until this mask is set and no timeout mechanism is implemented on HSM side.

The address of shared memory for Host-to-Hsm communication can be configured via the corresponding `vHsmUpdIpc` configuration parameter.



**Caution**
Hardware restrictions regarding the addresses for the communication Host-to-Hsm have to be respected by the user, i.e. alignment and/or locations have to be chosen correctly.

The vHsmUpd does provide a callout to start the Host system in the recovery mode. This shall be done in order to allow the system to trigger external watchdogs while the vHsmUpd updates the vHsm Fw. The callout should start the Host system. The callout can return at any point in time (e.g. immediately or when the host is ready), since the vHsmUpd waits explicitly until the host initialized the shared memory. A detailed description on how to implement the callout can be found in chapter 5.3.5.



**Note**
The Host systems application only must run out of RAM, when the Flash layout does not support Read-While-Write operations.

## 3.6 Main Functions

No main function available. All services are processed synchronously.



**Note**
The vHsmUpd calls MainFunctions of all used components itself, e.g. vMem and vMemAccM.

## 3.7 Structure of Update Header

The vHsmUpd uses a special statically defined structure to evaluate an update request. The format of this structure is shown in Figure 3-2. All contents are interpreted in big endian byte order.

| Update Header | |
|---|---|
| Tag | 16 Bytes to uniquely identify an update for the vHsm:<br>`0x56, 0x45, 0x43, 0x5F, 0x45, 0x50, 0x50, 0x5F,`<br>`0x48, 0x53, 0x4D, 0x00, 0x00, 0x00, 0x00, 0x01` |
| Version | 4 Byte to evaluate if update is needed due to old installed software image. |
| Address | 4 Byte address information where the update image is stored. |
| Length | 4 Byte length information to evaluate the size of the update image. This length shall include all data of the update image specified in Table 3-4. |
| CMAC | 16 Byte authentication information to verify the given image as valid. In the CMAC calculation all values from Table 3-4 are considered (in the order as mentioned within the table). Additionally, the version, address and length in the update header described in this table are included within the CMAC verification process. |

Table 3-2 Update Header

To validate the update header information a presence pattern value is used. The size of the presence pattern is determined by the flash page size (i.e. the smallest writable size in flash) and named Size_F. As the presence pattern must be page aligned, some platforms have to add padding information between the update header data and the presence pattern of the update header. The relative address of the tag to the presence pattern is therefore HW-dependent.

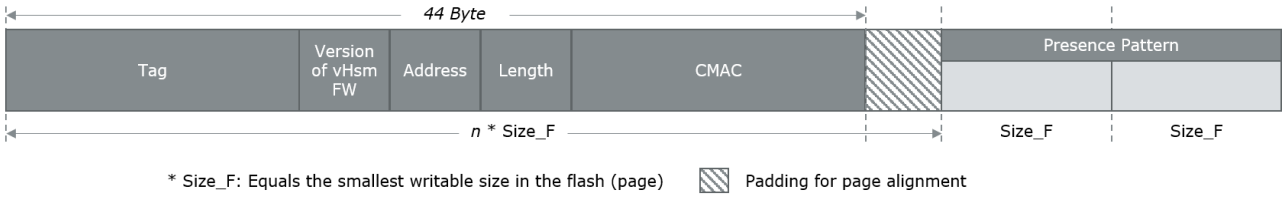



Figure 3-2 Update header format

Values of the presence pattern are shown in Table 3-3.

| Presence Pattern | |
|---|---|
| Presence Pattern value, i.e. first page | `0x73, 0x6A, 0x25, 0x3E` |
| Presence Pattern mask, i.e. second page (inverse value) | `0x8C, 0x95, 0xDA, 0xC1` |

Table 3-3 Presence Pattern

The presence pattern has three possible states:

- Validated, i.e. the first page of the presence pattern holds the pattern value and the second page is erased. This means the data within the update header is valid.
- Invalidated, i.e. the first page of the presence pattern holds the pattern value and the second page contains the pattern mask. The data is now invalid and will not be used for processing.
- Both pages erased or different page content, data will not be used.

**Caution**
The complete update header must be managed, i.e. erased and written (validated and invalidated) by Host-side, e.g. the Bootmanager. The vHsmUpd just evaluates given values but does never setup any write request (invalidation) to the Host-side. The vHsmUpd will recognize an update header as valid even after the update was correctly installed for a specific header.

## 3.8 Structure of Update Image

The vHsmUpd processes some parts of the update image. The necessary parts for the vHsmUpd are described in table Table 3-4. All the parts listed in the table are AES-128 CBC (no padding) encrypted.

© 2020 Vector Informatik GmbH

| Update Image | |
|---|---|
| Fw Offset | 4 Byte offset (count from this value as Offset 0) which indicates the start point of the vHsm Fw within the update image. |
| Crc Offset | 4 Byte offset which indicates the start point of the Crc value within the update image. |
| Padding | 8 Byte padding values. Value does not matter. |
| SBL | Additional 16 Byte aligned data, which will currently not be used by the vHsmUpd. Reserved for future use cases. |
| vHsm Fw | 16 Byte aligned firmware image. This data will be downloaded to the code area of the vHsm and later is the executable of the vHsm. |
| Crc | 4 Byte Crc32 value to verify the plain text data of the vHsm Fw. |
| Padding | 12 Byte padding values. Value does not matter. |

Table 3-4 Update Image

The vHsmUpd uses a constant Initialization Vector (IV) to decrypt the Update Image. Value for the IV is:

```
0x5370657A69616C4569576F6D69736175.
```

**Caution**
The value of the IV is only of interest when the Host side does not create the UpdateImage by using the vHsm, but the UpdateImage is created on its own.

The described update image of Table 3-4 is normally embedded in an update container, which is shown in the following image. The ContainerHeader parts and the Signature is only relevant for the Host side, i.e. the Bootmanager or the Flash-Bootloader.

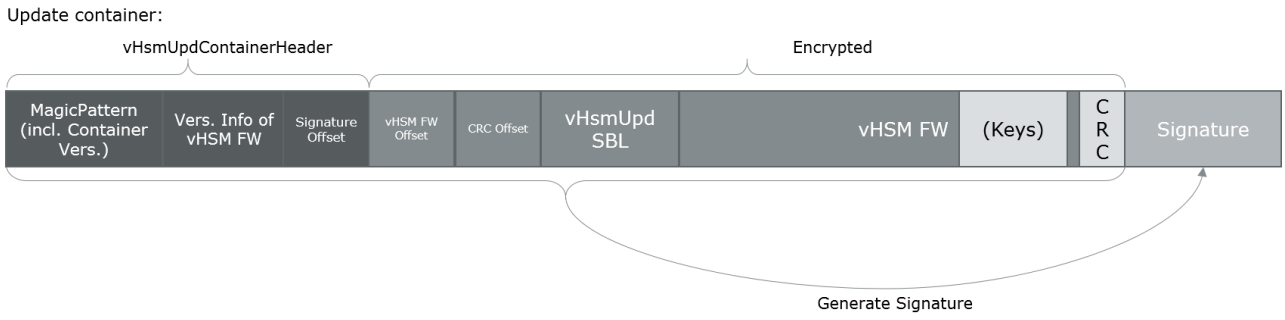



Figure 3-3 Update container with embedded update image (encrypted parts)

# 4 Integration

This chapter gives necessary information for the integration of the MICROSAR vHsmUpd into an application environment of an ECU.

## 4.1 Scope of Delivery

The delivery of the vHsmUpd contains the files which are described in the chapters 4.1.1 and 4.1.2:

### 4.1.1 Static Files

| File Name | Description |
|---|---|
| vHsmUpd_Crypto.c<br>vHsmUpd_Crypto.h<br>vHsmUpd_MemAb.c<br>vHsmUpd_MemAb.h<br>vHsmUpd_Ipc.c<br>vHsmUpd_Ipc.h | These are files for internal use of the vHsmUpd. |
| vHsmUpd_Hal.c<br>vHsmUpd_Hal.h | Files of the internal used Hal (Hardware Abstraction Layer). |
| vHsmUpd_Types.h | This file defines general types used by the vHsmUpd. |
| vHsmUpd_Callout.h | This file contains the definition of some callout functions. These functions have to be implemented by the user of the vHsmUpd. |
| vHsmUpd.c | This is the source file of the vHsmUpd. It contains the implementation of the main function. |
| vHsmUpd.h | This is the header file of the vHsmUpd. |

Table 4-1 Static files

### 4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator.

| File Name | Description |
|---|---|
| vHsmUpd_Cfg.h | This file contains the public configuration parameters of the vHsmUpd. |
| vHsmUpd_Cfg.c | This file contains configuration parameters and generated function calls to the vMemAccM and vMem used. Must be probably mapped to RAM, see Integration of Memory Stack. |

Table 4-2 Generated files

## 4.2 MemMap Sections used by vHsmUpd

The vHsmUpd uses several non-standard MemMap sections to control the flash layout. The used sections and how they are used is described within Table 4-3.

| MemMap section | |
|---|---|
| START_SEC_CONST_FIRMWAREPP<br>STOP_SEC_CONST_FIRMWAREPP | Section used for the presence pattern of the firmware. Must be directly located before the vHsm firmware and its size must be 3 pages. Part of HSM Flash. |
| START_SEC_CODE_MAC_CALLOUT<br>STOP_SEC_CODE_MAC_CALLOUT | Section used for the vHsm_FirmwareUpdate_GetMacKey_Callout |
| START_SEC_CODE_CIPHER_CALLOUT<br>STOP_SEC_CODE_CIPHER_CALLOUT | Section used for the vHsm_FirmwareUpdate_GetCipherKey_Callout |
| START_SEC_VAR_NOINIT_RAM_BUFFER<br>STOP_SEC_VAR_NOINIT_RAM_BUFFER | Section used for internal buffer which will be forwarded to the AES and the vMemAccM interface. On some platforms this buffer must be aligned (to prevent unaligned memory access exceptions). |
| START_SEC_VAR_NOINIT_RAM_BUFFER_TMP<br>STOP_SEC_VAR_NOINIT_RAM_BUFFER_TMP | Section used for internal buffer which will be forwarded to the vMemAccM interface.<br>On some platforms this buffer must be aligned (to prevent unaligned memory access exceptions).<br>Recommendation: Use the same alignment as for the above section. |
| START_SEC_CODE_RAM<br>STOP_SEC_CODE_RAM | Section used for generated function calls to vMemAccM and used vMem. Can be placed to RAM, when necessary due to write on program flash while executing code. |

Table 4-3 Used non-standard MemMap sections

## 4.3 Memory Layout of Secure Code Flash for HSM Firmware

The Hsm Updater is responsible for parts of the Hsm secure code area. The layout must be the way it is shown in Figure 4-1 for the Hsm Updater to recognize every part of the code area correctly.

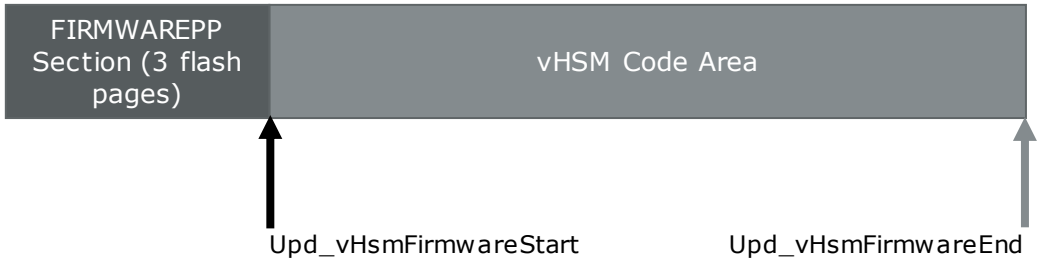



Figure 4-1 Memory Layout for Hsm Updater manage secure code area

The Hsm Updater uses two linker symbols, which must be defined within the build environment.

- `Upd_vHsmFirmwareStart`, shall be the first address of the flash area where the vHsm firmware image lies, i.e. the entry point of the vHsm firmware. The MemMap section `VHSMUPD_START_SEC_CONST_FIRMWAREPP` must directly lie before this symbol so the vHsmUpd can store version and validity information of the vHsm firmware to this section
- `Upd_vHsmFirmwareEnd`, shall be the last address of the flash area of the vHsm firmware.

**Caution**
All memory between the section `VHSMUPD_START_SEC_CONST_FIRMWAREPP` and the symbol `Upd_vHsmFirmwareEnd` is erased in order to write the new flash content to these locations.

Be aware that only complete flash blocks can be erased and that is the `VHSMUPD_START_SEC_CONST_FIRMWAREPP` and `Upd_vHsmFirmwareEnd` shall be flash block aligned.

The information about the flash blocks can be found in the hardware manual of the device used.

**Caution**
Some devices, e.g. ARM Cortex M3 cores, do not execute a jump to even addresses correctly. Therefore, the vHsmUpd allows an inconsistency of the least significant bit between the jump address, defined by the `Upd_vHsmFirmwareStart` symbol and the corresponding flash page start address, which is the next lower even address.

The `Upd_vHsmFirmwareStart` can be correctly mapped to the uneven start address of the vHsm Firmware, while the vHsmUpd will calculate with the lower even address to write to flash memory.

## 4.4 Device Startup

The Hsm Updater does not provide any startup code for a device. Should there be any need for it, the startup behavior must be implemented by the user and called before the Hsm Updater is executed. Startup code may be used to initialize MCU and PLL of the device.

## 4.5 Interrupt Vector Table

The Interrupt Vector Table (IVT) could be a part of vHsmUpd. Basically, there are two requirements to the IVT:

1) The Reset Vector points to the startup code of the vHsmUpd. This way vHsmUpd is able to decide whether to install a new version or start the currently installed version of the vHsm firmware.
2) All other interrupt vectors point to the vector table of the vHsm firmware. This means that the vector table of the firmware must be allocated statically at the same address.

Another possibility is to relocate the IVT during startup of the Hsm firmware. This allows a more flexible usage of the IVT since it can be omitted in the Hsm Updater.



**Caution regarding Interrupts**

The Hsm Updater does not need any interrupts for processing. In general, it is recommended to disable all interrupt sources during the Hsm startup/update sequence and only enable it later in the Hsm firmware.

Attention:

Some used vMem implementation could depend on enabled interrupts. If this is the case, just allow these interrupts to be executed. See [5] for information on interrupts needed.

## 4.6 Integration of Memory Stack for Secure Code Flash Access

The Hsm Updater uses a memory stack to erase and write to secure code flash area. On most hardware devices the parallel access to this flash area while the CPU executes instructions from it, is not allowed. This leads to several integration steps to be performed, when using a device with no parallel access possible.



**Note**

The info, whether a parallel access to secure code flash area from CPU and via the memory stack is allowed, can be found in the hardware manual of the used derivative. Normally it is described as RWW (read-while-write) partitions. If you cannot find any info about this topic in the hardware manual, contact the semiconductor of your device regarding this topic.

To solve the parallel access to the code flash area, all software parts, which are relevant for modifying memory access must be executed from RAM. This can be done, by mapping these components statically to RAM via the AUTOSAR memory mapping mechanism. It is important that all accessing parts are located in RAM, this includes code and constant data.

The following example shows how to map the vMemAccM to a specific linker section via the MemMap.h file for the Tasking compiler. All `#pragma` commands are compiler specific instructions and therefore not interchangeable between different compilers. See the manual of the compiler used for the specific command keyword to place code parts into its own linker section.



**Caution**
If even one part of the memory stack is not located in RAM and the software tries to modify the secure flash it will most likely result in a CPU exception/trap. Please check if all needed parts are mapped to RAM correctly.



**Example**

```
#if defined(VMEMACCM_START_SEC_CODE)
# undef VMEMACCM_START_SEC_CODE
# define START_SEC_CODE
# pragma section "vMemRamCode"
#endif

#if defined(VMEMACCM_STOP_SEC_CODE)
# undef VMEMACCM_STOP_SEC_CODE
# define STOP_SEC_CODE
# pragma endsection
#endif

#if defined(VMEMACCM_START_SEC_CONST_UNSPECIFIED)
# undef VMEMACCM_START_SEC_CONST_UNSPECIFIED
# define START_SEC_CONST_UNSPECIFIED
# pragma section "vMemData"
#endif

#if defined(VMEMACCM_STOP_SEC_CONST_UNSPECIFIED)
# undef VMEMACCM_STOP_SEC_CONST_UNSPECIFIED
# define STOP_SEC_CONST
# pragma endsection
#endif
```

The vMemRamCode and vMemData sections then can be defined in the used linker script to contain all data which shall be executed to RAM. This linker section can be mapped to a RAM location. Normally the compiler generated startup code will later copy the RAM code from ROM to RAM and always executed the RAM code. Please ensure this in your environment for proper functionality.

This must be repeated for the following software parts (each part is involved in accessing secure code flash):

- vMemAccM (shown in example)
- vMem
- vHsmUpd_Hal
- vHsmUpd_Cfg.c



**Caution**
Remember to place ALL constant data sections AND code sections to RAM.

# 5 API Description

The vHsmUpd does not provide any interface to other modules.

## 5.1 Type Definitions

The types defined by the vHsmUpd are described in this chapter.

| Type Name | C-Type | Description | Value Range |
|---|---|---|---|
| vHsmUpd_FirmwareVersionType | uint32 | Type used for the firmware information of the vHsm. | 0 – 4294967295 |
| vHsmUpd_IpcUpdateStateType | uint8 | Type used for the current state of the vHsmUpd which is communicated to the Host | VHSMUPD_UPDATE_NOT_STARTED (0x1)<br>VHSMUPD_UPDATE_IN_PROGRESS (0x2)<br>VHSMUPD_UPDATE_COMPLETED (0x3) |

Table 5-1 Type definitions

## 5.2 Services used by vHsmUpd

In the following table services provided by other components, which are used by the vHsmUpd are listed. For details about prototype and functionality refer to the documentation of the providing component.

| Component | API |
|---|---|
| Crc | Crc_CalculateCRC32 |
| vMem | vMem_Init |
| | vMem_MainFunction |
| vMemAccM | vMemAccM_Init |
| | vMemAccM_MainFunction |
| | vMemAccM_Write |
| | vMemAccM_Erase |
| vSecPrim | esl_initWorkSpaceHeader |
| | esl_initDecryptAES128 |
| | esl_decryptAES128 |
| | esl_finalizeDecryptAES128 |
| | esl_initCMACAES128 |
| | esl_updateCMACAES128 |
| | esl_finalizeCMACAES128 |

Table 5-2 Services used by the vHsmUpd

## 5.3 Callout Functions

At its configurable interfaces the vHsmUpd defines callout functions. The declarations of the callout functions are provided by the vHsmUpd . It is the integrator's task to provide the corresponding function definitions. The definitions of the callouts can be adjusted to the system's needs. The callout function declarations are described in the following tables:

### 5.3.1 vHsm_FirmwareUpdate_GetMacKey_Callout

| **Prototype** | |
|---|---|
| `Std_ReturnType` **`vHsm_FirmwareUpdate_GetMacKey_Callout`**<br>`(uint8* keyPtr, uint32* length)` | |
| **Parameter** | |
| `keyPtr [out]` | keyPtr where the key is stored. |
| `Length [in,out]` | length available for the key that shall be set to the length used. |
| **Return code** | |
| `Std_ReturnType` | E_OK if a valid key was found, E_NOT_OK otherwise. |
| **Functional Description** | |
| The vHsmUpd verifies the given update image by generating a CMAC. The user shall provide the key for this task, which will be used for the calculation. | |
| **Particularities and Limitations** | |
| Hint: The CMAC is also built by the vHsm firmware during download of the new image. So, the same key must be used by the vHsm firmware. This callout function can be mapped to a designated address within ROM, to which the vHsm firmware may jump to get the same key for its CMAC calculation. | |
| Call context | |
| > TASK<br>> This function is Non-Reentrant | |

Table 5-3 vHsm_FirmwareUpdate_GetMacKey_Callout

### 5.3.2 vHsm_FirmwareUpdate_GetCipherKey_Callout

| **Prototype** | |
|---|---|
| `Std_ReturnType` **`vHsm_FirmwareUpdate_GetCipherKey_Callout`**<br>`(uint8* keyPtr, uint32* length)` | |
| **Parameter** | |
| `keyPtr [out]` | keyPtr where the key is stored. |
| `Length [in,out]` | length available for the key that shall be set to the length used. |
| **Return code** | |
| `Std_ReturnType` | E_OK if a valid key was found, E_NOT_OK otherwise. |
| **Functional Description** | |
| The vHsmUpd decrypts the given update image by using a user-specific key, which must be provided by this callout. | |

| Particularities and Limitations |
| --- |
| Hint: The encryption is executed by the vHsm firmware during download of the new image. So, the same key must be used by the vHsmUpd to decrypt the image again. This callout function can be mapped to a designated address within ROM, to which the vHsm firmware and the vHsmUpd may jump to get the same key for the encryption and decryption job. Alternative: implement the same callout twice, within the vHsm firmware and the vHsmUpd. |
| Call context |
| > TASK<br>> This function is Non-Reentrant |

Table 5-4 vHsm_FirmwareUpdate_GetCipherKey_Callout

### 5.3.3 vHsm_FirmwareUpdate_Error_Callout

| Prototype | |
| --- | --- |
| `void` **`vHsm_FirmwareUpdate_Error_Callout`**<br>`(const vHsmUpd_ErrorType error)` | |
| **Parameter** | |
| `error [out]` | error, which indicates the kind of error happened within the vHsmUpd. |
| **Return code** | |
| - | Callout does not provide a Return code as this function should not return to the vHsmUpd context. |
| **Functional Description** | |
| The function is called when an uncorrectable error occurred.<br>This can happen on two occasions. The update failed too often or there is no update image available, but also no valid vHsm firmware image. In both cases the callout should not return to normal execution as this leads to an endless loop. Could be used to start the Host-cores without secure boot to do more advanced error processing. | |
| **Particularities and Limitations** | |
| - | |
| Call context | |
| > TASK<br>> This function is Non-Reentrant | |

Table 5-5 vHsm_FirmwareUpdate_Error_Callout



**Caution**
The `vHsm_FirmwareUpdate_Error_Callout` shall not return to normal execution of the application but do error handling and may issue a reset of the device or starting the application cores to do a more sophisticated error handling. If the callout returns the vHsm core will just execute an endless loop, but not trigger any other functionality.

### 5.3.4 vHsm_FirmwareUpdate_VersionCompare_Callout

| Prototype | |
|---|---|
| `void` **`vHsm_FirmwareUpdate_VersionCompare_Callout`** `(const uint8* versionBuffer, const uint16 dataLength)` | |
| **Parameter** | |
| `versionBuffer [in]` | Buffer, which holds the data for the version comparison. |
| `dataLength [in]` | Length of the data within the versionBuffer. |
| **Return code** | |
| `boolean` | Shall return TRUE if an update of the firmware is requested and FALSE otherwise. |
| **Functional Description** | |
| The function is called when the vHsmUpd wants to determine if the installed firmware version is outdated. The version-buffer contains the value read from flash, which can be used by the function to check, whether an update shall be requested or not. | |
| **Particularities and Limitations** | |
| Only relevant if VHSMUPD_ENABLE_USER_COMPARE_MECHANISM is set to STD_ON. Otherwise a decimal comparison is done between the two firmware versions. | |
| Call context | |
| > TASK<br>> This function is Non-Reentrant | |

Table 5-6 vHsm_FirmwareUpdate_VersionCompare_Callout

### 5.3.5 vHsm_FirmwareUpdate_Update_Callout

| Prototype | |
|---|---|
| `Std_ReturnType` **`vHsm_FirmwareUpdate_Update_Callout`** `(void)` | |
| **Parameter** | |
| void | none |
| **Return code** | |
| Std_ReturnType | E_OK if host core is prepared for update process, E_NOT_OK otherwise. |
| **Functional Description** | |
| If in recovery mode an update is requested, this callout is invoked before the actual update process is started. The user has at this state the possibility e.g. to start a host core which triggers watchdogs. vHsmUpd will not check the state of the "application" which then runs on the Host, e.g. problems with the Host application in general are not recognized by the vHsmUpd itself but shall be evaluated within this callout.<br>Only when the callout returns success with value E_OK the update will be processed, return value of E_NOT_OK cancels the update process. | |
| **Particularities and Limitations** | |
| - | |

| Call context |
|---|
| > TASK<br>> This function is Synchronous<br>> This function is Non-Reentrant |

Table 5-7 vHsm_FirmwareUpdate_Update_Callout



**Caution**
If needed by the system, e.g. since an external watchdog must be triggered, the callout must start the Host system. When the callout returns, the vHsmUpd assumes a correct working Host application which does allow the vHsmUpd to proceed with updating the vHsm Fw.

This callout will be only invoked in the vHsmUpd recovery mode.

### 5.3.6 vHsmUpd_FirmwareUpdate_TriggerSoftwareReset_Callout

| Prototype | |
|---|---|
| `void vHsm_FirmwareUpdate_TriggerSoftwareReset_Callout (void)` | |
| **Parameter** | |
| void | none |
| **Return code** | |
| - | Callout does not provide a Return code as this function should not return to the vHsmUpd context. |
| **Functional Description** | |
| If the precondition is fulfilled, the user shall implement this callout to reset the device.<br>A reset of the device is triggered in case that the vHsm firmware is invalid before the update. This callout shall not return to the vHsmUpd context. The Hsm platforms' function vHsm_Hal_TriggerSoftwareReset() can be used as a guide for the implementation of this callout. | |
| **Particularities and Limitations** | |
| This callout is only triggered if VHSMUP_HAL_ENABLE_RESET is not defined by the hardware abstraction layer. | |
| Call context | |
| > TASK<br>> This function is Synchronous<br>> This function is Non-Reentrant | |

Table 5-8 vHsmUpd_FirmwareUpdate_TriggerSoftwareReset_Callout



**Note**
The vHsmUpd checks whether `VHSMUPD_HAL_ENABLE_RESET` is defined. If it is not defined, the reset is not supported by the used hardware abstraction layer and the reset callout is called.

If the reset is supported by the hardware abstraction layer, this callout can be left empty, since not used.

# 6 Configuration

In the vHsmUpd the attributes can be configured according to/ with the following methods/ tools:

- Configuration with DaVinci Configurator Pro

## 6.1 Configuration Variants

The vHsmUpd supports the configuration variants

- `VARIANT-PRE-COMPILE`

## 6.2 Configuration with DaVinci Configurator Pro

The configuration classes of the vHsmUpd parameters depend on the supported configuration variants. For their definitions please see the vHsmUpd_bswmd.arxml file.

### 6.2.1 vHsmUpd Header

There are two possible configuration scenarios available:

- Use two dedicated fixed addresses for header location (`vHsmUpdUpdateHeaderFixAddress`).
- Use a start address and a configurable number of possible header addresses (`vHsmUpdUpdateHeaderVariableAddress`). The first address shall point to the first update header. All additional header addresses are calculated internally and therefore all headers shall be located consecutively in the flash memory with no gaps in between.



**Caution**
The address to configure for the Update Header shall be the presence pattern of the Update Header and NOT the first user-data. The structure of the update header can be found in chapter 3.7. Be aware of needed padding on some platforms!

### 6.2.2 vHsmUpd Retry Strategy using vHsmUpdRetryCounter

The vHsmUpd can retry a failed update a configurable number of times. After the update fails completely the callout described in 5.3.3 will be invoked. An update failure results in erasing the flash and restarting of writing the image to flash. All progress of the update will be lost.

Be aware that during a retry of an update the new flash image will not be verified again. This implicates that `vHsm_FirmwareUpdate_GetMacKey_Callout` to get the key for verifying the encrypted update image will not be called again. Furthermore, the `vHsm_FirmwareUpdate_Update_Callout` to start the Host will also not be called. Still

the `vHsm_FirmwareUpdate_GetCipherKey_Callout` to get the key for decrypting the update image will be called another time as the image decryption must start from the beginning.



**Note**
Simple retries of write and erase operations can be configured within the vMemAccM. These retries do not influence the behavior of the vHsmUpd. See [3] for details.

### 6.2.3 vMemAccM and vMem Configuration Restrictions

The vHsmUpd and the vMemAccM validate most of the configuration. For a detailed description of the vMemAccM and vMem parameters and their configuration see [3], [4] and [5].

General approach shall be:

- Use only one vMem
- Respect Table 6-1

| vMemAccM configuration | |
|---|---|
| vMemAccMAddressAreaConfiguration | ID = 0 |
| | ReadRequestLength is obsolete for the vHsmUpd. |
| | Priority = 0 |
| vMemAccMAddressAreaVariantConfigurations | Do not configure variants here. |
| vMemAccMSubAddressAreaConfigurations | Use physical address for the virtual start address. |

Table 6-1 vMemAccM configuration



**Caution**
The above-mentioned settings are mandatory and cannot be omitted.



**Note**
All parameters like DET, MainFunction triggering are not relevant in context of vHsmUpd and do not have effects and can be disabled.



**Recommendation**
Configure only one vMemAccM SubAddressArea which covers the complete flash block used, i.e. the HSM code flash area. This also implicates that only one vMemSector needs to be configured.

## 6.3 Configuration of Hardware Abstraction Layer (HAL)

The HAL of the Hsm Updater does not provide a configuration interface. Some HALs of the Hsm Updater component provide a configuration file named vHsmUpd_Hal_Cfg.h, which allows a manual configuration. This has only impact on the functionality of the HAL itself and does not have any impact on the Hsm Updater itself.

For details on how to configure the HAL for your use case, look within the configuration header file provided.



**Caution**
If the vHsmUpd_Hal_Cfg.h is not available within your SIP the HAL does not need to be configured. Simply ignore this chapter then.

# 7 Additional Hints and Recommendations Testing the vHsmUpd

This chapter shall help the to create “common” test cases which should be executed to verify the update process.

- Test not only the update itself, but test that the newly installed image also can be updated once again.
- If the vHsmUpdEnableUserCompareMechanism configuration parameter is `STD_OFF` the vHsmUpd does a 32-Bit value comparison – if an update is requested - between the currently installed version information and the update version. Therefore, the number of updates the vHsmUpd can execute is limited. When the value `0xFFFFFFFF` is reached no more updates of the vHsm Fw are allowed and possible. Consider this in your update design.
- Do Worst-Case stress tests during the update mechanism to verify that the update preparation and execution, e.g. creation of the update container via Software Download, can be done even while the device is under full load. Additional reset tests during update are also recommended to verify expected behavior on Host and HSM side.
- Check the consistency between your different projects, especially between vHsm Fw, vHsmUpd and the Host Bootmanager/FlashBootLoader. The Host and the vHsm Fw are responsible for creating the update image for the vHsmUpd. The vHsmUpd must recognize the update and parse it correctly. The consistency should be given when all possible scenarios/use cases for the interaction are tested at least once:
  - No update, i.e. normal startup of device (on Host and HSM), with no update image available and with update image available.
  - Update creation (Host Application + vHsm Fw).
  - Update of vHsm Fw with vHsmUpd on two cases, with vHsm Fw available or not available.

© 2020 Vector Informatik GmbH

# 8 Glossary and Abbreviations

## 8.1 Glossary

| Term | Description |
|---|---|
| Flash Block | A flash block is a continuous memory area, which is the smallest erasable unit in a flash memory. |
| Update Header | Static defined memory layout area, which is needed by the vHsmUpd to evaluate and perform an update request. |
| Update Image | A downloaded/executable vHsm firmware image, which will be processed, i.e. flashed to the vHSM code area. |
| Size_F | The smallest writable unit within flash memory, also called a flash page. |

Table 8-1 Glossary

## 8.2 Abbreviations

| Abbreviation | Description |
|---|---|
| AES | Advanced Encryption Standard |
| API | Application Programming Interface |
| AUTOSAR | Automotive Open System Architecture |
| BSW | Basis Software |
| ECU | Electronic Control Unit |
| HAL | Hardware Abstraction Layer |
| HSM | Hardware Security Module |
| ISR | Interrupt Service Routine |
| IV | Initialization Vector of AES cryptoalgorithm |
| IVT | Interrupt Vector Table |
| MICROSAR | Microcontroller Open System Architecture (the Vector AUTOSAR solution) |
| PLL | Phase-Locked Loop |

Table 8-2 Abbreviations

# 9 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com